Learning Project – OutreachHub – MEAN Stack

Project Execution Plan

No	Topic	Activity	Duration
1	Git & GitHub	Manage source code with Git and GitHub	1 hours
2	HTML	Build core structure of authentication, home, and contacts modules	6 hours
3	CSS & SCSS	Style authentication, home, and contacts modules	12 hours
4	Javascript	Add interactivity to authentication and contacts modules using pre-built APIs	16 hours
5	MongoDB	Define database schemas and write queries	8 hours
6	Node.js	Build backend APIs for all modules	24 hours
7	Typescript	Convert some javascript code to typescript	4 hours
8	NestJS	Convert complete backend code to NestJS	32 hours
9	Angular	Build complete frontend and integrate backend APIs	56 hours

Project Summary

It is a SaaS platform designed for businesses to manage their contacts, create simple message templates, and simulate targeted campaigns. The project will progressively build features as you learn core full-stack technologies, resulting in a full web application.

- It provides distinct portals,
    - Admin Portal: for admin users to manage business workspaces, workspace users
    - OutreachHub: for workspace users to control their workspace's contacts, templates, and campaigns
- Admin users can,
    - Manage business workspaces and workspace users
- Workspace users can perform actions on-behalf of the business,
    - Manage contacts with tagging
    - Manage message templates like "text" and "text & image"
    - Manage and launch campaigns for selected tags of contacts and selecting templates, and view campaign status

Project Definition

1. Implement multi-tenant (workspaces) platform comprising of two major parts,
    a. Admin portal: For managing business workspaces and workspace users
    b. OutreachHub portal: For workspace users with features like contacts, message templates, campaigns, analytics
    c. Admin portal
        i. Only admin users can access this portal.
        ii. Modules:
            1. Authentication module
                a. Login and Logout
            2. Workspaces module
                a. List, create, view, update, and delete workspaces
                b. Workspace users sub-module
                    i. List, create, view, update, and delete workspace users
    d. OutreachHub portal (User facing portal):
        i. Only workspace users can access this portal.
        ii. Workspace user Roles:
            1. Editor: Can perform all operations on all modules
            2. Viewer: Can perform only read operations on all modules
        iii. Modules:
            1. Authentication module
                a. Login and Logout
            2. Home module
                a. Welcome message
                b. Workspace analytics
                    i. Charts:
                        1. Number of campaigns done per day filterable by date range
                        2. Number of campaign messages sent per message type per day filterable by date range
                        3. Number of contacts reached per day filterable by date range
                    ii. Tables:
                        1. List of recent 5 campaigns with targeted contacts tags

2. List of top 5 tags with highest number of contacts

3. **Contacts module**
    a. List, create, view, update, and delete contacts
    b. Contacts can have multiple tags (tags are used in campaigns to target specific sub-set of contacts)
    c. Note: Contacts will be primarily identified by phone number.
4. **Message templates module**
    a. List, create, view, update, and delete message templates
    b. Message templates will have two types "Text" and "Text & Image".
5. **Campaigns module**
    a. List, create, view, update, delete, copy campaigns
    b. The newly created campaign will be in draft state.
    c. Separate launch campaign action to actually run a campaign.
        i. Virtually run a campaign and store campaign messages details per individual contact in database.
        ii. Note: Storing information per message in a database is crucial as contact or template can be updated in future, but we must store actual data at the time of sending message.
        iii. Show live campaign status changing from draft → running → completed. Use polling mechanism to get updated status from backend to frontend.
    d. Only draft campaigns can be updated. Launched campaigns cannot be updated.
    e. Add copy campaign action to duplicate a campaign with the same configurations.

e. **Additional Use Case:** Improve the implementation for the case where a single user with the same email address is added to more than one workspace. How will the user access multiple workspaces from the OutreachHub portal?
f. **Note:** Maintain all code in GitHub account for preventing any loss and managing the progress across the project lifecycle.

Git & GitHub

1. **Activity:** Manage source code with Git and GitHub
2. **Note:** Maintain all code in GitHub account for preventing any loss and managing the progress across the project lifecycle.
3. **Definition:**
    a. Perform some actions to implement Git and GitHub knowledge in following order:
        i. Create a git repo on your GitHub account.
        ii. Clone repo in local environment.
        iii. Create a "readme.md" file with the information about the project in the "main" branch.
        iv. Commit and push the "readme.md" file.
        v. Create a branch with the purpose of changing some code. Use an appropriate name for the branch.
            1. Modify the "readme.md" file's content.
            2. Create a new "index.html" file with dummy content.
            3. Commit to new changes to the new branch.
            4. Push the new branch to GitHub.
            5. Create a pull request from new branch to "main" branch on GitHub.
            6. Share your repo access with a colleague on GitHub and ask them to review your pull request.
            7. Once a pull request is approved by a colleague, Merge the pull request.
            8. Checkout to the "main" branch in local environment.
            9. Get the new merged changes from the remote repository using git commands.
            10. Delete the previously created branch for changes from both local and remote environments.
        vi. Do the same changes using the fork method and submit changes using a pull request instead of creating a new branch in the same repo.

HTML

1. **Activity:** Build core structure of authentication, home, and contacts modules
2. **Definition:**

a. User HTML to create pages for different modules and actions.
b. Authentication module
    i. Create login page that navigates to home page on login action.
    ii. Add logout button in navigation menu which takes user to login page.
c. Create navigation menu with links to all feature modules
    i. Add logout button that navigates to login page
d. Home module
    i. Add navigation menu with links to other pages and logout action with appropriate navigation
    ii. Add welcome message with some branding image
    iii. Add placeholder images for analytics charts
    iv. Create tables for other table-based analytics
e. Contacts module
    i. List contacts
    ii. View contact details
    iii. Create a new contact
    iv. Update a contact
    v. Delete a contact
f. General Considerations:
    i. Navigate user to details page on add or update actions
    ii. Navigate user to list page on delete action
    iii. Add pagination in lists to prevent performance issues when actual data is rendered using APIs in future activities.

CSS & SCSS

1. **Activity:** Style authentication, home, and contacts modules
2. **Definition:**
    a. Add CSS styles to previously built HTML structure for authentication, home, and contacts modules
    b. Make sure to use responsive design principles to clean user interface on all screen sizes from large screens to mobile screens.
    c. Use SCSS for writing styles instead of CSS.

Javascript

1. **Activity:** Add interactivity to authentication and contacts modules using pre-built APIs

2. **Definition:**
    a. Use provided pre-built APIs setup to integrate APIs using fetch method.
    b. Use Javascript DOM manipulation techniques to display the data.
    c. Use Javascript to handle:
        i. CRUD API calls
        ii. Programmatic navigation when required
        iii. Handling form submission
        iv. Browser-side data storage of authentication credentials
    d. Implement some modules for workspace users with "Editor" and "Viewer" roles
        i. Login module
            1. Use login API
        ii. Navigation menu
            1. Logout API
        iii. Contacts module
            1. List contacts: Use list contacts API with pagination
            2. View contact details: Use contact details API
            3. Create a new contact: Use create contact API
            4. Update a contact: Use update contact API
            5. Delete a contact: Use delete contact API
    e. General considerations:
        i. Implement authentication and authorization using JWT token returned by Login API.
        ii. Build list pages with pagination and default sorting on name or timestamp columns to prevent performance issues.
        iii. Show confirmation popup for delete actions.

MongoDB

1. **Activity:** Define database schemas and write queries
2. **Definition:**
    a. Implement a multi-tenant (workspace-wise) database structure by scoping the data with workspace ids wherever applicable.
        i. Admin users
        ii. Workspaces
        iii. Workspace users
        iv. Contacts
        v. Message Templates

vi. Campaigns
vii. Campaign Messages

b. List out and write queries for all the modules and actions defined in the project definition.
c. Create appropriate indexes for optimizing read queries.

Node.js

1. **Activity:** Build backend APIs for all modules
2. **Definition:**
    a. Create backend APIs for all the modules and actions defined in the project definition for Admin portal and OutreachHub portals.
    b. Create a postman collection for developing and testing all the APIs.
    c. Use standard HTTP Methods and Response Codes.
    d. Take inspiration from the pre-built APIs used in the previous activity in javascript section and learning materials.
    e. Replace previously used pre-built APIs with newly built Node.js APIs in the frontend (HTML-CSS-JS) code.

Typescript

1. **Activity:** Convert some javascript code to typescript
2. **Definition:**
    a. Convert given Javascript Code to Typescript Code
    b. Compile the Typescript Code
    c. Run the Typescript Code

NestJS

1. **Activity:** Convert complete backend code to NestJS
2. **Definition:**
    a. Convert previously created Node.js-Express APIs code to NestJS code.
    b. Use Standard NestJS practices learned during the learning to build the APIs.

Angular

1. **Activity:** Build complete frontend and integrate backend APIs
2. **Definition:**

a. Implement the complete Admin portal and OutreachHub portal frontend as described in the Project Definition section.
b. Integrate backend APIs built in previous section (NestJS).